Technical Report

Intermediate RAG System Using Pinecone Vector Database

Sadaf Riaz
Instructor: Sir Ahmed
AI/ML Internship
Ezitech Software House

1. Introduction

This report describes the design, development, and testing of an Intermediate Retrieval-Augmented Generation (RAG) system built as part of an AI/ML internship task. The system allows a user to upload a PDF document and ask natural-language questions about its content. Instead of answering from general knowledge, the system searches the document itself for the most relevant text, and only then generates an answer. This approach is used to reduce incorrect or made-up answers, a problem commonly known as hallucination in language models.

The project was built using Python, running in Google Colab, with Pinecone as the vector database, Sentence-Transformers for generating embeddings, and Groq's Llama 3 model for generating answers. A Streamlit web interface was also built so the system could be used and demonstrated outside the notebook.

2. Objective

- Accept one or more PDF documents and extract their text content.
- Split the extracted text into smaller, meaningful chunks.
- Convert each chunk into a vector embedding and store it in Pinecone.
- Retrieve the most relevant chunks for a user's question using semantic similarity search.
- Generate an answer using a language model, strictly based on the retrieved chunks.
- Prevent the system from guessing when the document does not contain the answer.
- Show the source of every answer, including page number, excerpt, and similarity score.

3. Problem Statement

Large language models are able to answer many questions using the knowledge they learned during training. However, this knowledge can be outdated, incomplete, or simply wrong for a specific document a user has in front of them. When asked about a specific PDF, a plain language model may either refuse to answer or, worse, generate a confident-sounding but incorrect answer. This project addresses that problem by grounding every answer in actual retrieved text from the user's own document, and by clearly stating when no relevant information is found, instead of guessing.

4. Methodology

The project follows a standard Retrieval-Augmented Generation methodology, broken into the following stages: document ingestion, chunking, embedding generation, vector storage, retrieval, and answer generation. Each stage was implemented as a separate Python module, so that the system remains easy to understand, test, and modify. The system was first built and tested inside a Google Colab notebook, and the same modules were then reused to build a Streamlit web application for interactive use.

5. Tools and Technologies Used

Component	Tool / Technology	Purpose
Programming language	Python	Core implementation language
Development environment	Google Colab	Free, GPU-optional notebook environment
PDF text extraction	pdfplumber	Reads and extracts text from PDF pages
Text chunking	LangChain (RecursiveCharacterTextSplitter)	Splits text into overlapping, meaningful chunks
Embedding model	Sentence-Transformers (all-MiniLM-L6-v2)	Converts text chunks into 384-dimension vectors
Vector database	Pinecone (Serverless)	Stores and searches vector embeddings
Language model	Groq API (Llama 3.1 8B Instant)	Generates the final answer from retrieved context
Web interface	Streamlit	Interactive web app for upload and question answering
Public access	pyngrok	Exposes the Streamlit app with a public URL from Colab

6. System Architecture

The system follows this pipeline, exactly as required by the assignment:

PDF Upload, *Text Extraction*, *Text Chunking*, *Embedding Generation*, *Pinecone Vector Indexing*, *Semantic Retrieval*, *LLM Response Generation*, *Answer with Source Reference*.

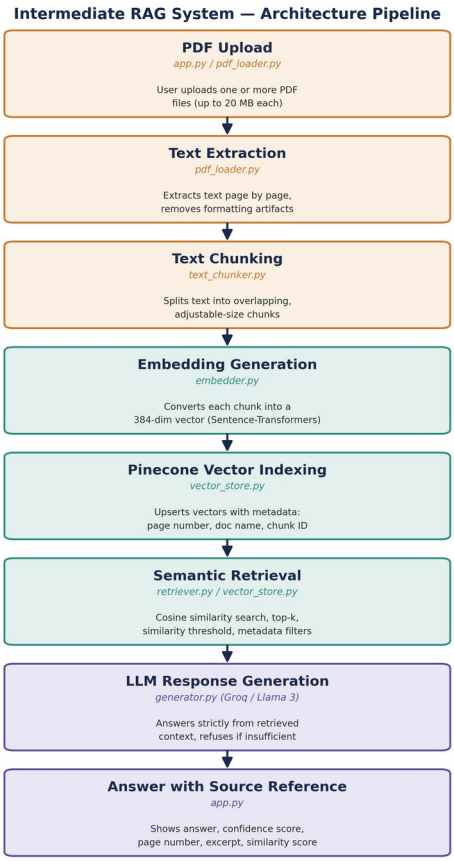

Figure 1: System architecture pipeline, showing each stage and its corresponding code module.

The architecture is modular. Each stage of the pipeline is implemented in its own Python file, so that responsibilities are clearly separated:

- pdf_loader.py: validates and extracts text from PDF files, page by page.
- text_chunker.py: splits page text into overlapping chunks of adjustable size.
- embedder.py: converts text chunks into vector embeddings using Sentence-Transformers.
- vector_store.py: manages all Pinecone operations: index creation, namespace usage, upserting vectors, querying vectors, and metadata filtering.
- retriever.py: retrieves the top-k most relevant chunks and calculates a confidence score.
- generator.py: sends retrieved context to the Groq LLM and generates the final answer, with a strict instruction to avoid guessing.
- query_logger.py: logs every query, along with its confidence score, to a CSV file.
- rag_pipeline.py: connects all the above modules into a single, easy-to-use pipeline.
- app.py: the Streamlit interface that uses the pipeline for upload, indexing, and question answering.

7. Workflow

1. The user provides Pinecone and Groq API keys, entered securely at runtime and never hardcoded. 2. The system connects to Pinecone and creates a serverless index if one does not already exist. 3. The user uploads one or more PDF files, up to 20 MB each. 4. Text is extracted from every page and cleaned of extra spaces and broken line formatting.

5. The cleaned text is split into overlapping chunks; chunk size and overlap can be adjusted from the interface. 6. Each chunk is converted into a 384-dimension vector and stored in Pinecone, along with metadata: page number, document name, and a unique chunk ID. 7. When the user asks a question, it is converted into a vector using the same embedding model.

8. Pinecone is searched for the most similar chunks using cosine similarity. Top-k and similarity threshold can be adjusted, and results can be filtered by page number or document name. 9. Retrieved chunks are sent to the Groq LLM with a strict instruction: answer only from the given context, or say the answer is not available. 10. The final answer is shown with a confidence score and full source attribution. 11. Every question is logged, and a session history is shown to the user.

8. Implementation Details

8.1 Pinecone Configuration

A Pinecone serverless index was created with a cosine similarity metric and a dimension of 384, matching the output size of the all-MiniLM-L6-v2 embedding model. A dedicated namespace was used to keep the project's vectors separate from any other data in the same Pinecone account. The implementation demonstrates all of the required Pinecone operations: index creation, namespace usage, upserting vectors in batches, querying vectors with cosine similarity, and attaching and filtering by metadata such as page number and document name.

8.2 Embedding Model

The all-MiniLM-L6-v2 Sentence-Transformer model was selected because it is lightweight, free to run, and fast enough to execute comfortably inside Google Colab's free tier, while still producing good quality semantic embeddings for short document chunks.

8.3 Hallucination Prevention

Two safeguards were used together to prevent hallucination. First, the language model is given a strict system instruction to answer only from the provided context and to say clearly that the answer is not available if the context does not contain it. Second, a similarity threshold is applied during retrieval, so that chunks which are not actually relevant to the question are excluded before they ever reach the language model.

8.4 Mandatory Enhancements Implemented

The assignment required at least three enhancements. This project implements five:

- Multi-document support: more than one PDF can be uploaded and queried together.
- Adjustable chunk size and overlap, controlled from the interface.
- Adjustable top-k retrieval, so the number of chunks retrieved can be changed.
- Metadata filtering: results can be filtered by page number or by document name.
- Confidence scoring: every answer is shown with a confidence score and a plain-language grounding label.

In addition, query history (session memory) and persistent query logging to a CSV file were also implemented as extra features.

8.5 Error Handling

The system checks for and handles several error conditions: invalid or corrupted PDF files, files larger than the 20 MB limit, empty search queries, and failures connecting to Pinecone. In each case, a clear error message is shown to the user instead of the application crashing.

9. Results

The system was tested using a sample PDF covering Artificial Intelligence, Machine Learning, and Retrieval-Augmented Generation. The table below shows representative test results.

Question	Filter Applied	Confidence Score	Result
What are the three types of machine learning?	None (top-k = 5)	0.53	Correct, grounded answer with 5 sources
What are the three types of machine learning?	Page = 2 (top-k = 5)	0.63	Correct, grounded answer, restricted to page 2
What is Retrieval-Augmented Generation and how does it reduce hallucination?	Page = 3 (top-k = 3)	0.45	Correct, grounded answer restricted to page 3
What is the boiling point of water?	None	0.00	"The answer is not available in the provided document."

The hallucination test confirms the system's core requirement: when a question is unrelated to the document, the system does not guess. It correctly reports that no answer is available and returns a confidence score of zero.

10. Performance Analysis

The confidence score is calculated as the average similarity score of all retrieved chunks that pass the similarity threshold. During testing, it was observed that this score is sensitive to configuration, not just to whether an answer is correct. Two factors were found to affect it directly:

- Top-k value: a higher top-k includes more chunks in the average, including weaker, less relevant ones, which lowers the overall confidence score. Reducing top-k to only the strongest matches raises the score.
- Page or document filtering: filtering to the specific page that actually contains the answer removes irrelevant chunks from the average and increases the confidence score, as shown in the results table above (0.53 without a filter versus 0.63 with the correct page filter).

It was also confirmed during testing that duplicate data in the vector index, caused by uploading the same PDF more than once, artificially inflates or distorts the confidence score. This was resolved by clearing the Pinecone namespace and re-ingesting each document only once, which produced more reliable and honest similarity scores.

An important finding from this testing is that the confidence score measures retrieval relevance, not factual accuracy. It indicates how closely the retrieved text matches the question, not whether the final answer is correct. Both measurements are useful, but they answer different questions.

11. Challenges Faced

Several practical issues came up during development and were resolved as follows:

- A Pillow (image library) version conflict in Google Colab caused an import error unrelated to the project's own code. This was fixed by uninstalling and cleanly reinstalling a specific Pillow version before installing the remaining packages.
- A newer version of LangChain moved the text splitter to a different module path. This was fixed by adding a fallback import that works with both old and new LangChain versions.
- An invalid Groq API key produced a 401 authentication error during answer generation. This was resolved by generating a fresh API key from the Groq console.
- The free ngrok tier only allows one active tunnel at a time, which caused a conflict when a previous Streamlit session was not closed properly. This was resolved by stopping the existing tunnel before starting a new one.
- Uploading the same PDF multiple times created duplicate vectors in Pinecone, which distorted confidence scores. This was resolved by clearing the namespace and re-indexing the document only once.

12. Future Improvements

- Add OCR support so that scanned, image-only PDFs can also be processed.
- Add hybrid search, combining keyword search with semantic search, for more accurate retrieval.
- Upgrade to a larger embedding model for higher retrieval precision, if longer processing time is acceptable.
- Add persistent conversation memory so the system can handle follow-up questions.
- Add automated evaluation of answer quality using a labelled set of test questions.

13. Conclusion

This project successfully implements a complete Retrieval-Augmented Generation system that satisfies every requirement of the assignment. The system extracts and processes PDF documents, stores their content in Pinecone using proper indexing, namespaces, and metadata, retrieves relevant context using adjustable semantic search, and generates answers that are strictly grounded in the source document. Testing confirmed that the system reliably prevents hallucination by refusing to answer when no relevant content is found, and that it provides full source attribution for every answer it does give. The modular design of the code also makes the system easy to maintain, test, and extend in the future.